PRÁCTICA UD1 · EXPLOTACIÓN Y DESPLIEGUE DE UNA APLICACIÓN CON IA

Testia

Apuntes → test de repaso con IA, desplegado en Vercel (serverless) y en AWS EC2 (Docker), protegido con Clerk.

Juan Amador

Trabajo individual · GitHub amador-h

testia.vercel.app

54.235.82.94

github.com/amador-h/testia

LA MINIAPP

Una pantalla, un formulario y una única operación de IA

Qué hace

Pegas unos apuntes, eliges nº de preguntas (3–10) y dificultad, y obtienes un test de opción múltiple con la explicación de cada respuesta.

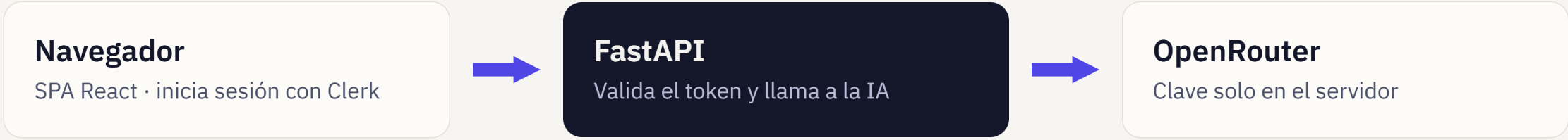
FastAPI valida la entrada, llama a OpenRouter (modelos gratuitos) y devuelve JSON validado con Pydantic.

Qué no hace, a propósito

- Sin base de datos: los apuntes no se guardan
- Sin historial, chat ni gestión de usuarios propia
- Sin paneles ni pantallas adicionales
- La identidad se delega en Clerk

ARQUITECTURA

El navegador nunca habla con la IA



	Vercel	AWS EC2
Backend	Vercel Function por petición	uvicorn permanente en un contenedor
SPA	CDN de Vercel	Servida por FastAPI
Secretos	Variables de entorno de Vercel	Archivo privado --env-file (600)
Transporte	HTTPS incluido	HTTP: solo cuentas de prueba

VERCEL

Un único proyecto: SPA en la CDN y FastAPI como función

pyproject.toml declara el entrypoint de FastAPI; Vercel lo ejecuta con el runtime Python.

vercel.json compila la SPA y fija maxDuration a 60 s.

.vercelignore impide subir .env y cachés.

Comando de la CLI	Para qué
vercel link	Vincular el proyecto único
vercel env add --sensitive	Secretos por stdin
vercel env ls	Nombres, nunca valores
vercel deploy	Preview
vercel curl	Probar la Preview protegida
vercel deploy --prod	Publicar en testia.vercel.app
vercel logs / inspect	Diagnosticar

AUTENTICACIÓN CON CLERK

Ocultar el botón no protege: FastAPI valida cada token

1

La SPA inicia sesión con Clerk (instancia de desarrollo)

2

getToken() y envío como Authorization: Bearer

3

El SDK oficial comprueba firma (JWKS), caducidad y azp

4

Sin token válido: 401 y la IA no se llama

```
$ curl -i -X POST https://testia.vercel.app/api/generar --data-binary @cuerpo.json
HTTP/1.1 401 Unauthorized · Www-Authenticate: Bearer
{"detail":"Necesitas iniciar sesión para generar un test."}
```

SECRETOS Y CONFIGURACIÓN

Solo la clave pública de Clerk llega al navegador

Variable	Tipo	Vercel	EC2	Cuándo
OPENROUTER_API_KEY	Secreto	env (sensitive)	testia.env (600)	Ejecución
CLERK_SECRET_KEY	Secreto	env (sensitive)	testia.env (600)	Ejecución
CLERK_AUTHORIZED_PARTIES	Config	por entorno	http://54.235.82.94	Ejecución
OPENROUTER_MODEL / TIMEOUT	Config	env	testia.env	Ejecución
VITE_CLERK_PUBLISHABLE_KEY	Pública	env (build)	--build-arg	Compilación

Nada en Git: solo .env.example con valores ficticios.

Nada en la imagen: los secretos se inyectan al ejecutar.

Nada en comandos: los valores entran por stdin.

DOCKER

Una imagen, un contenedor, la aplicación completa

Etapa 1 · node:24-alpine

npm ci + npm run build con la clave pública como build-arg

Etapa 2 · python:3.12-slim

FastAPI sirve /api/* y la SPA compilada (180 MB)

Usuario sin privilegios (app) y HEALTHCHECK
.dockerignore excluye .env, .git y tests

uvicorn en 0.0.0.0:8000, publicado en el host
Etiqueta = commit: se sabe qué versión corre

```
docker run -d --name testia --env-file .env -p 8080:8000 --restart unless-stopped testia:<commit>
```

AWS EC2

Todo creado con la AWS CLI, solo lo necesario abierto

Recurso	Valor
Instancia	t3.micro · us-east-1b · running
Sistema	Amazon Linux 2023 · x86_64
IP elástica	54.235.82.94 (fija)
Docker	25.0 · arranca con el sistema
Clave SSH	ED25519 propia, solo en mi PC
Variables	/opt/testia/testia.env · 600

Security Group testia-sg

Puerto	Origen	Para qué
22 TCP	Mi IP /32	SSH de administración
80 TCP	0.0.0.0/0	Web de la demo

Cerrados y comprobados desde fuera: 8000 (interno del contenedor) y 2375 (API de Docker).

La imagen se construye en EC2 desde el tag v1.0-aws del repositorio público y corre con --restart unless-stopped en el puerto 80.

OPERACIÓN

Estado, reinicio y actualización comprobados

Estado y logs

docker ps muestra healthy; docker logs registra el motivo de cada rechazo, nunca tokens ni apuntes.

Reinicio

docker restart y reinicio completo de la instancia: el contenedor vuelve solo.

Actualización

git checkout del tag, build, sustituir el contenedor. La imagen anterior permite volver atrás.

```
antes: testia:c1347c2
después: testia:55ccc4d (tag v1.0-aws) Up (healthy) 0.0.0.0:80→8000/tcp
/api/health → 200 · /api/generar sin token → 401
```

UNA INCIDENCIA REAL

401 con sesión iniciada, solo en Docker local

Síntoma	Con sesión en localhost:8080 los primeros envíos fallaban; en Vercel funcionaba.
Diagnóstico	docker logs: TOKEN_NOT_ACTIVE_YET. El reloj de Windows, y el del contenedor, iba unos 34 s atrasado: el nbf del token quedaba en el futuro.
Decisión	Sincronizar la hora, sin ampliar el margen de validación del token, para no relajar la seguridad. En EC2 el reloj va por NTP.
Comprobación	POST /api/generar → 200 OK en los logs del contenedor.

HISTORIAL GIT

Un tag y una release por hito

v0.1	v0.2	v0.3	v0.4	v0.5	v1.0
Backend y 4 tests	SPA en local	Vercel	Clerk y redespliegue	Docker local	AWS EC2

Trabajo individual

Todos los commits son de la cuenta amador-h: código, configuración, pruebas, diagnóstico y documentación.

Trazabilidad

Cada entorno indica su commit desplegado; la imagen Docker lleva el commit como etiqueta.

DEMO EN VIVO

Los dos entornos, con sesión y sin ella

Vercel · testia.vercel.app

1. Sin sesión: el botón pide iniciar sesión
2. Iniciar sesión y generar un test
3. Cerrar sesión
4. curl sin token: 401
5. Dashboard: proyecto y variables

EC2 · 54.235.82.94

1. Iniciar sesión y generar un test
2. curl sin token: 401
3. SSH: docker ps y docker logs
4. docker restart y volver a probar
5. Consola: Security Group e instancia

Plan B ante un fallo externo: capturas y salidas de consola en evidencias/ y en el README.

GRACIAS

¿Preguntas?

Repositorio	`github.com/amador-h/testia`
Versión final	`v1.0-aws · 55ccc4d`
Vercel	`https://testia.vercel.app`
AWS EC2	`http://54.235.82.94`